

PLANO DE ENSINO – PÓS-GRADUAÇÃO LATO SENSU

1. IDENTIFICAÇÃO		
Código e Nome da Disciplina PG2305-04-Z251 – Desenvolvimento de software integrado - DevOps		Carga horária 24h
Pré-requisito Conhecimentos básicos de desenvolvimento de software, programação, Git/linha de comando, APIs, redes, containers e noções de ambientes de execução	Curso/Turma PG2305-04-Z251	Centro CCT
Dias de Aulas 10, 11, 12, 24, 25 e 26 de setembro de 2026		Semestre 2026.2

2. EMENTA	
Práticas DevOps aplicadas ao desenvolvimento de software integrado, com foco em colaboração, automação, qualidade, segurança e entrega confiável de uma aplicação. Cultura DevOps, fluxo de valor, feedback e métricas de entrega. Versionamento com Git, revisão de código, rastreabilidade e colaboração em equipe. Integração contínua com build, testes automatizados, análise de qualidade e geração de artefatos. Containerização e integração de serviços com Docker e Docker Compose. Entrega contínua com ambientes, configuração, segredos, gates de qualidade e segurança, versionamento de releases e rollback. Uso de inteligência artificial generativa e agentes de codificação no ciclo de desenvolvimento, incluindo apoio a requisitos, análise de código, geração/refatoração, testes, revisão de pull requests, documentação, troubleshooting e análise de logs, com validação humana, segurança, privacidade e governança. Observabilidade aplicada com logs, métricas, alertas, resposta a incidentes e melhoria contínua. Noções de infraestrutura como código, DevSecOps, GitOps, engenharia de plataformas, AIOps e automação inteligente como tendências de evolução do pipeline DevOps.	

3. OBJETIVOS DE APRENDIZAGEM DA DISCIPLINA	
1. Explicar os princípios DevOps, o fluxo de valor de software e a relação entre cultura, automação, qualidade e operação. 2. Diagnosticar gargalos, tarefas manuais, riscos de integração e oportunidades de automação em um fluxo de desenvolvimento de software. 3. Estruturar um fluxo colaborativo com Git, branches, pull requests, revisão de código, tags, releases e rastreabilidade de mudanças. 4. Configurar pipelines de integração contínua para build, testes automatizados, análise de qualidade e geração de artefatos. 5. Empacotar e integrar componentes de software com Docker e Docker Compose, incluindo variáveis de ambiente, serviços dependentes, volumes e health checks. 6. Planejar um fluxo de entrega contínua com ambientes, critérios de promoção, versionamento, configuração, segredos, rollback e gates de segurança. 7. Aplicar ferramentas de IA generativa de forma crítica e segura para apoiar análise, implementação, testes, revisão, documentação e diagnóstico de software. 8. Utilizar logs, métricas, alertas, dashboards, SLOs e métricas DORA para avaliar entregas, diagnosticar falhas e apoiar melhoria contínua.	



9. Entregar e apresentar um projeto integrador demonstrando conexão entre repositório, pipeline, testes, containers, uso responsável de IA, release, operação e proposta de evolução DevOps.

4. CONTEÚDO	CH 24h
UNIDADE I – Fundamentos DevOps, Fluxo de Valor e IA no SDLC • DevOps como abordagem sociotécnica: colaboração, comunicação, responsabilidade compartilhada e foco no cliente. • Ciclo de vida DevOps: planejamento, código, build, testes, release, deploy, operação, monitoramento e feedback. • Fluxo de valor, gargalos, tarefas manuais, riscos de integração e melhoria contínua. • Métricas DORA, diagnóstico inicial de maturidade DevOps e visão de IA generativa como apoio ao ciclo de desenvolvimento.	4h
UNIDADE II – Versionamento, Colaboração, Qualidade e IA Assistida • Git, repositórios remotos, branches, pull requests, code review, tags e releases. • Estratégias de branching, integração frequente e rastreabilidade de mudanças. • Padronização de projeto, documentação mínima, convenções, linters, análise estática e gestão de dependências. • Uso de IA para compreensão de código, refatoração assistida, documentação, revisão de pull requests e apoio à tomada de decisão técnica.	4h
UNIDADE III – Integração Contínua, Testes e Automação com IA • Conceitos de CI, runners, jobs, etapas, artefatos, cache, variáveis e gatilhos de build. • Pipeline com build, testes unitários, testes de integração, cobertura e gates de qualidade. • IA para geração de casos de teste, análise de falhas de pipeline, explicação de logs e sugestões de correção. • Publicação de artefatos ou imagens e preparação técnica para release, mantendo revisão humana e execução de testes como critério de aceite.	4h
UNIDADE IV – Containers e Integração de Serviços • Dockerfile, imagens, camadas, registry e boas práticas de empacotamento. • Docker Compose para aplicações multi-serviço, redes, volumes, variáveis de ambiente e health checks. • Integração entre API, banco de dados, serviços auxiliares e ambiente local reproduzível. • Apoio de IA para leitura de erros, documentação de comandos e melhoria de arquivos Dockerfile/compose.yaml, com validação por execução local.	4h
UNIDADE V – Entrega Contínua, Configuração, Segurança e Governança de IA • Conceitos de CD, promoção de artefatos, versionamento semântico e estratégias de deploy/rollback. • Gestão de ambientes, configuração externa, variáveis, segredos e permissões mínimas. • Gates de qualidade e segurança: dependências, imagens, análise estática, critérios de promoção e validação de código gerado por IA. • Guardrails para IA no desenvolvimento: proteção de segredos, dados sensíveis, privacidade, licenças, prompt injection, hallucination, overreliance e revisão humana.	4h
UNIDADE VI – Observabilidade, IA Aplicada e Projeto Integrador • Logs, métricas, traces, dashboards, alertas, SLI/SLO e validação operacional de releases. • Uso de IA para análise de logs, troubleshooting, resumo de incidentes, hipóteses de	4h



causa raiz e proposta de correção.

- Ferramentas possíveis conforme disponibilidade: GitHub Copilot, ChatGPT/Codex, Cursor, Gemini Code Assist, Amazon Q Developer ou equivalentes.
- Apresentação do projeto integrador com repositório, pipeline, testes, containers, registro crítico do uso de IA, release, observabilidade e proposta de evolução DevOps.

5. METODOLOGIA DE ENSINO DA DISCIPLINA

A disciplina será conduzida por meio de aulas expositivas dialogadas, demonstrações técnicas e laboratórios práticos, com evolução progressiva de um fluxo realista de desenvolvimento integrado. A abordagem priorizará a construção de um projeto integrador, no qual os estudantes aplicarão versionamento, integração contínua, testes, containerização, entrega, segurança básica, uso responsável de IA e observabilidade.

Os estudantes realizarão diagnóstico DevOps, questões-problema, análise de gargalos, exercícios de automação e proposta de melhoria para um cenário de desenvolvimento e operação de software.

O módulo de IA será tratado de forma aplicada: os estudantes poderão usar assistentes de IA para compreender código, propor testes, revisar mudanças, documentar decisões, interpretar logs e apoiar troubleshooting, sempre com validação humana, execução de testes e revisão de segurança.

A cada encontro serão produzidas evidências técnicas, como commits, pull requests, logs de pipeline, relatórios de teste, imagens/artefatos, arquivos de configuração, registros de execução e, quando houver uso de IA, registro de prompts, contexto fornecido, saída aproveitada, validações e ajustes realizados.

As discussões sobre maturidade, GitOps, engenharia de plataformas e AIOps serão tratadas como contextualização para evolução do pipeline, sem substituir o foco prático da disciplina.

As atividades valorizarão colaboração, rastreabilidade, automação, qualidade, segurança básica, responsabilidade no uso de IA e capacidade de explicar decisões técnicas adotadas ao longo do ciclo de vida do software.

6. AVALIAÇÃO DA DISCIPLINA

A avaliação será contínua e prática, baseada na capacidade de aplicar conceitos DevOps ao desenvolvimento, integração, entrega e operação de uma aplicação em ambiente controlado.

Critérios de avaliação:

- Projeto integrador funcional, com repositório organizado, aplicação executável, documentação técnica mínima e registro crítico do uso de IA quando aplicado (25%).
- Pipeline de integração contínua com build, testes automatizados, análise de qualidade e geração/publicação de artefatos ou imagens (25%).
- Containerização e integração de serviços, incluindo Dockerfile, Docker Compose, variáveis de ambiente, volumes, health checks e instruções de execução (20%).
- Evidências de entrega e operação: release, rollback ou plano de rollback, configuração, segurança básica, logs, métricas/dashboards e análise de falhas (15%).
- Diagnóstico DevOps e proposta de evolução de práticas de automação, incluindo oportunidades de uso de IA com riscos, limites e guardrails (10%).
- Apresentação final com defesa das decisões de arquitetura, pipeline, qualidade, segurança, automação, uso de IA e operação (5%).

Evidências avaliativas:

- Link ou pacote do repositório com histórico de commits, pull requests, tags/releases e documentação.
- Arquivos de pipeline, Dockerfile, compose.yaml, scripts, configurações e registros de execução.
- Relatórios ou evidências de execução de testes, análise de qualidade, build, publicação de imagem/artefato e execução da aplicação.
- Registro do diagnóstico inicial, principais problemas encontrados, decisões adotadas, análise operacional e melhorias propostas.
- Quando houver uso de IA: problema tratado, prompt/contexto utilizado, saída aproveitada, validações

executadas, correções humanas e riscos identificados.

@uniforcomunica

7. BIBLIOGRAFIA

Básica

HUMBLE, Jez; FARLEY, David. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley Professional, 2010.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John; FORSGREN, Nicole. The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations. 2. ed. Portland: IT Revolution Press, 2021.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. Portland: IT Revolution Press, 2018.

FARLEY, David. Modern Software Engineering: Doing What Works to Build Better Software Faster. Boston: Addison-Wesley Professional, 2021.

Complementar

NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.

MORRIS, Kief. Infrastructure as Code. 2. ed. Sebastopol: O'Reilly Media, 2020.

BEYER, Betsy; JONES, Chris; PETOFF, Jennifer; MURPHY, Niall Richard (org.). Site Reliability Engineering: How Google Runs Production Systems. Sebastopol: O'Reilly Media, 2016. Disponível em: <https://sre.google/sre-book/>. Acesso em: 1 set. 2026.

DORA; GOOGLE CLOUD. DORA's software delivery performance metrics. Disponível em: <https://dora.dev/guides/dora-metrics/>. Acesso em: 1 set. 2026.

GITHUB. GitHub Actions documentation. Disponível em: <https://docs.github.com/en/actions>. Acesso em: 1 set. 2026.

DOCKER. Docker Compose documentation. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 1 set. 2026.

HASHICORP. Terraform Documentation. Disponível em: <https://developer.hashicorp.com/terraform/docs>. Acesso em: 1 set. 2026.

GITHUB. GitHub Copilot documentation. Disponível em: <https://docs.github.com/en/copilot>. Acesso em: 1 set. 2026.

OPENAI. Codex: AI coding agents for software engineering. Disponível em: <https://openai.com/codex/>. Acesso em: 1 set. 2026.

GOOGLE. Gemini Code Assist overview. Disponível em: <https://developers.google.com/gemini-code-assist/docs/overview>. Acesso em: 1 set. 2026.

NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Gaithersburg: National Institute of Standards and Technology, 2023. Disponível em: <https://doi.org/10.6028/NIST.AI.100-1>. Acesso em: 1 set. 2026.

OWASP. OWASP GenAI LLM Top 10 2026. Disponível em: <https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>. Acesso em: 1 set. 2026.

8. ELABORADOR(ES)

Nome do Professor da Disciplina: Arimatéia Júnior

Currículo Resumido: Profissional com mais de 15 anos de experiência em TI, com atuação em Infraestrutura, Cloud, DevOps, Arquitetura e Engenharia de Software. Atua em desenho e evolução de plataformas, automação de processos e práticas de documentação técnica para times de engenharia. Possui certificações em Microsoft Azure, AWS, Google Cloud, Oracle, GitHub e VMware.

Endereço do Currículo (Lattes ou LinkedIn): <https://www.linkedin.com/in/arimateiajunior>